Personalized Course Recommendation System

Comparing five AI approaches with a prerequisite-aware hybrid recommender

Project Report

CS24308 Artificial Intelligence

Third Year, Computer Science

Team member	Name	Roll number
Member 1		
Member 2		
Member 3		
Member 4		
Member 5		

Submitted to: _____

November 2026

Abstract

Choosing electives is difficult because the best choice depends on a student's interests, career goal, completed prerequisites, year level and ability, constraints that ordinary movie-style recommenders ignore. This project builds a personalized course recommendation system in Python that ranks the top five courses a student is eligible for and likely to rate highly in the next semester. We compare five approaches, namely a Popularity baseline, content-based filtering with TF-IDF, user-based k-nearest-neighbour collaborative filtering, and matrix factorization trained with Alternating Least Squares, plus a Random floor, on synthetic but rule-based data of 600 students, 48 courses and about 11,500 ratings. Our own contribution, the Hybrid PCHR, blends the model scores, adds career-goal fit and a difficulty-versus-CGPA penalty, enforces prerequisites and re-ranks with Maximal Marginal Relevance. Using a leave-last-semester-out protocol on five random datasets, the hybrid reaches an NDCG@5 of 0.805 against 0.713 for the best single model, winning on all five datasets. An ablation shows that the eligibility rule and the career/interest profile account for most of the gain. A command-line demo gives explained recommendations and supports new students.

Contents

1. Introduction and problem statement
2. Background
3. Dataset and environment
4. Methodology
5. Models compared
6. Proposed improvement: Hybrid PCHR
7. Evaluation metrics
8. Experiments and results
9. Demonstration
10. Discussion, limitations and ethics
11. Conclusion and future work
12. Team contributions
13. References
14. Appendix: running the code

1. Introduction and problem statement

Every semester a student must pick electives from a catalogue. A poor choice wastes time, lowers grades and delays progress in a chosen career direction. Advisers help, but they cannot give every student individual attention. A recommender system can: it learns from the ratings of past students and from the student's own history to suggest courses likely to fit.

Course choice differs from recommending films or products in three ways. First, **prerequisites**: Deep Learning cannot be taken before Machine Learning. Second, **year-level gating**: courses open in later semesters only. Third, **difficulty**: a demanding course fits a strong record better than a weak one. A model that ignores these rules will often recommend courses the student cannot take.

1.1 Problem formulation

Given a student's rated course history, CGPA, career goal, declared interests and target semester, rank all courses the student has not yet taken and return the top K = 5. A suggestion counts as correct when the student actually took the course in the held-out semester and rated it 4 or more out of 5.

1.2 Objectives

- Implement and compare several AI approaches to the same task under one fair protocol.
- Design an improvement that handles the real constraints of course selection and measure which parts of it help.
- Report quantitative results with several metrics, an ablation study and a cold-start experiment.
- Provide a working demonstration with explanations for each recommendation.

1.3 Scope

The system recommends electives from a fixed catalogue of 48 courses for a single student at a time. It does not handle timetable clashes, credit limits or seat availability. Data is synthetic, generated from documented rules, because real enrolment data is private.

2. Background

Content-based filtering recommends items similar to those a user liked, measured on item descriptions. Text is commonly represented with TF-IDF weights [3], which raise the weight of words that are frequent in a document but rare across the collection. **Collaborative filtering** uses the behaviour of many users; the neighbourhood approach predicts a rating from the ratings of the most similar users [1]. **Matrix factorization** approximates the rating matrix by the product of low-rank user and item factors and was a leading method in large recommendation benchmarks [2]. **Hybrid recommenders** combine several techniques to offset the weaknesses of each [9].

Top-N recommendation is evaluated like information retrieval. Precision, recall and Normalized Discounted Cumulative Gain (NDCG) [5] measure how many relevant items are found and how high they are ranked. Because a list of near-identical items is of little use, diversity-aware re-ranking such as Maximal Marginal Relevance (MMR) [4] selects each new item for relevance and dissimilarity to those already chosen. General coverage of the field is given in [6]. The framing of our system as a utility-based learning agent follows [8].

3. Dataset and environment

3.1 Why synthetic data

Real enrolment records contain private grades and ratings. We therefore generate data with explicit behavioural rules. This gives known ground truth, reproducible experiments through fixed random seeds, and a controlled setting for an ablation. The code reads and writes three CSV files, so real data with the same columns can replace ours.

3.2 Data generation

- **Catalogue:** 48 courses over four years (8, 12, 16 and 12 courses), six topics (AI/ML, Data, Systems, Web/Software, Security, Theory), difficulty 1 to 5 and explicit prerequisite lists.
- **Students:** each has a branch (CSE, IT, ECE, AI&DS), a hidden interest vector drawn from a Dirichlet distribution tilted by branch, a career goal correlated with the interests, a CGPA (normal, mean 7.6, clipped to 5-9.8) and 4 to 6 completed semesters. A declared-interest survey lists the top two topics, with one of the two replaced by a random topic for 25 percent of students.
- **Choices:** each semester a student chooses 3 to 5 courses from those that are eligible, with probability increasing in interest match, career fit, course quality, level fit and decreasing in difficulty above ability. Sampling uses the Gumbel-top-k trick [10] at temperature 1.3 so that choices are biased but not deterministic.
- **Ratings:** rating = 2.3 + 2.2 x interest + 0.8 x career fit + course quality - 0.45 x (difficulty above ability) + noise (standard deviation 0.7), rounded and clipped to 1 to 5.

Item	Value (demonstration dataset, seed 42)
Students / courses	600 / 48
Ratings	11,578 (19.3 per student, range 12 to 27); 40.2% of the student-course matrix observed
Rating distribution	1: 5.1%, 2: 21.1%, 3: 30.1%, 4: 23.5%, 5: 20.2% (mean 3.33)
Branches	CSE 253, AI&DS 152, IT 110, ECE 85
CGPA	mean 7.66, standard deviation 0.90
Completed semesters	4: 174 students, 5: 239 students, 6: 187 students
Training / test ratings	9,212 / 2,366 (leave-last-semester-out)
Evaluated students	496 of 600 have at least one relevant (rated 4+) test course; mean 2.16 relevant courses
Eligible pool per student	9.5 courses on average (minimum 3, maximum 16)

Table 1. Dataset statistics. The five experiment datasets use seeds 0 to 4 and have the same size and similar statistics.

3.3 Environment

Python 3 with numpy, pandas, scikit-learn (TF-IDF and cosine similarity only) and matplotlib. Matrix factorization, nearest-neighbour prediction, the hybrid, the evaluation protocol and all metrics are written by hand. The whole program is one file, `course_recommender.py`.

4. Methodology

4.1 Train/test protocol

We use **leave-last-semester-out**: for every student, the courses taken in the final completed semester are hidden as the test set and all earlier semesters form the training data. This copies real use (recommend the next semester) and keeps prerequisite chains intact. A random split would put a student's later courses in training and leak the future. For tuning, the same procedure is applied one semester earlier, so the hybrid's weights are chosen on a validation semester and the test set is never used for selection.

4.2 Eligibility rule

A course is eligible for a student in target semester s if (a) it has not been taken, (b) its year level is at most ceil(s / 2) and (c) every prerequisite is completed. In code, with prerequisite matrix P and the 0/1 vector of taken courses, condition (c) is P x taken >= number of prerequisites.

4.3 Two comparison settings

In the **raw** setting every not-yet-taken course is a candidate, so models that ignore the rules may recommend courses the student cannot take. In the **fair** setting every model may recommend only eligible courses, so the remaining differences come from ranking quality and the extra information a model uses. Both are reported.

4.4 Repetition

The whole pipeline, including data generation and tuning, is repeated on five datasets (seeds 0 to 4) and reported as mean and standard deviation. K = 5 is used throughout.

5. Models compared

5.1 Random and Popularity (baselines)

Random ranks courses by random numbers; it is the floor that any real model must clearly beat. **Popularity** recommends the courses taken by most students. For rating prediction it uses a damped item mean, (sum of ratings + 5 x global mean) / (count + 5), so that a course with one 5-star rating does not look perfect.

5.2 Content-based filtering (TF-IDF)

Each course is described by its name, description and topic label. The text is converted to TF-IDF vectors and every pair of courses is compared with cosine similarity, cos(a, b) = a . b / (|a| |b|). A student's profile weights each rated course by max(rating - 2, 0), so liked courses count and disliked ones do not. The score of a candidate is its weighted average similarity to the student's rated courses. The model needs no other students and can score a new course, but it recommends look-alikes and its score is not a rating.

5.3 User-based kNN collaborative filtering

Ratings are mean-centred per student so that generous and harsh raters are comparable. Similarity between the target student and every other student is the cosine of the centred rating vectors (missing entries are zero). The 30 most similar students with positive similarity are used, and the prediction is

```
predicted = mean(student) + sum( sim x deviation ) / ( sum( sim ) + 2 )
```

where deviation is a neighbour's rating minus that neighbour's mean. The constant 2 is a shrinkage term that keeps the prediction near the student's mean when only a few neighbours rated the course, so one enthusiastic neighbour cannot dominate. The student's own row is excluded.

5.4 Matrix factorization (ALS)

The model is rating = mu + b_{item} + $b_{student}$ + p . q, with 8 latent factors per student and course. Item and student biases are computed once with shrinkage 5. Factors are learned by Alternating Least Squares: fixing the course factors, each student's vector is the solution of a small ridge regression; then the roles are swapped; 15 rounds with regularization 4. Each step has a closed form, so no learning rate is needed. A student not in the training matrix, or one with few ratings, is folded in by one ridge regression against the fixed course factors. Predictions are clipped to the range 1 to 5.

6. Proposed improvement: Hybrid PCHR

PCHR stands for Prerequisite-, Career- and difficulty-aware Hybrid Recommender with MMR re-ranking. The idea is that each base model sees a different signal (rating habits, similar students, course text, popularity), while the student's profile and the catalogue rules add information that none of them use.

6.1 Scoring

For each student, the outputs of the four models are standardised (z-scores) so that ratings, similarities and counts share a scale, and combined:

```
score = w_mf*z(MF) + w_cf*z(kNN) + w_cb*z(Content) + w_pop*z(log popularity)
      + w_profile*profile_fit - w_diff*max(0, difficulty - ability)
```

- **profile_fit** combines the weight of the course topic in the student's career goal (scaled 0 to 1) and a 0/1 match with the declared interests.
- **ability** = 1 + (CGPA - 5) / 5 x 4 maps CGPA 5 to 10 onto the 1 to 5 difficulty scale.
- **Eligibility**: ineligible courses are removed before ranking.
- **MMR**: the list is built greedily by choosing the course with the highest (1 - gamma) x relevance - gamma x (largest TF-IDF similarity to courses already chosen), with gamma = 0.15.
- **Tuning**: a grid search over 96 weight combinations on the validation semester maximises NDCG@5. On seed 0 it chose w_mf 0.5, w_cf 0.5, w_cb 0, w_pop 0, w_profile 1.0, w_diff 0.3.
- **Explanations**: for each recommended course the system lists which signals fired: career or interest fit, similar courses the student rated highly, similar students' ratings, completed prerequisites and difficulty versus CGPA.

6.2 Cold start

When a student has no ratings, the rating-based terms are zero and the profile fit, popularity and eligibility rule decide, so the hybrid can still give personalised, valid recommendations.

6.3 The system as an agent

The recommender is a utility-based learning agent [8]: its sensors are the rated history, CGPA, career goal and interests; it scores every course with a numeric utility; it learns taste from ratings and tunes its weights; and its actuator is a ranked list with reasons. The environment is partially observable, stochastic and episodic.

7. Evaluation metrics

Metric	Definition	Reads as
Precision@K	hits in top K / K	cleanliness of the list; higher is better
Recall@K	hits in top K / number of relevant courses	share of liked courses found
NDCG@K	DCG / ideal DCG, DCG = sum of rel(i) / log2(i + 1)	quality including rank order; main ranking metric
Hit rate	1 if at least one hit in top K, else 0	lenient; saturates easily
Coverage	distinct courses recommended to anyone / 48	use of the catalogue; low means popularity bias
Topic spread	distinct topics in a list / list length	variety within one list
Eligible rate	eligible recommendations / total	practicality of the advice
RMSE, MAE	square root of mean squared error; mean absolute error of predicted ratings	rating accuracy; lower is better

Table 2. Metrics. A held-out course is relevant if its rating is 4 or more. RMSE and MAE are computed only for models that predict a 1 to 5 rating.

Worked example: if the relevant courses are at ranks 1 and 3 and there are two relevant courses in total, then DCG = 1 + 1/log2(4) = 1.5, ideal DCG = 1 + 1/log2(3) = 1.631 and NDCG = 0.92. With K = 5 and about 2.0 relevant courses per evaluated student, the maximum average precision is about 0.40.

8. Experiments and results

All values are means over five datasets of 600 students, K = 5. Standard deviations across datasets are typically 0.01 to 0.03.

8.1 Raw setting: all untaken courses are candidates

Model	Precision	Recall	NDCG	Hit	Coverage	Topic spread	Eligible
Random	0.065	0.165	0.119	0.290	0.971	0.731	0.311
Popularity	0.129	0.354	0.241	0.490	0.554	0.661	0.898
Content-Based (TF-IDF)	0.173	0.421	0.349	0.604	0.800	0.550	0.494
User-kNN CF	0.258	0.648	0.567	0.799	0.667	0.630	0.609
Matrix Factorization	0.242	0.606	0.507	0.764	0.688	0.599	0.565
Hybrid PCHR	0.366	0.924	0.805	0.975	0.742	0.641	1.000

Table 3. Raw comparison. Baselines often recommend courses the student cannot yet take (eligible rate 0.49 to 0.61), which partly explains the large gap.

8.2 Fair setting: every model recommends eligible courses only

Model	Precision	Recall	NDCG (mean +/- sd)	Hit	Coverage	Topic spread
Random	0.208	0.544	0.382 +/- 0.017	0.757	0.746	0.750
Popularity	0.153	0.417	0.275 +/- 0.032	0.576	0.650	0.692
Content-Based (TF-IDF)	0.283	0.716	0.565 +/- 0.023	0.860	0.725	0.672
User-kNN CF	0.325	0.831	0.713 +/- 0.016	0.934	0.742	0.695
Matrix Factorization	0.331	0.849	0.708 +/- 0.017	0.941	0.738	0.675
Hybrid PCHR	0.366	0.924	0.805 +/- 0.014	0.975	0.742	0.641

Table 4. Fair comparison (main result). The hybrid beats the best single model, kNN, by 0.091 NDCG on average (minimum +0.079) and wins on all 5 datasets.

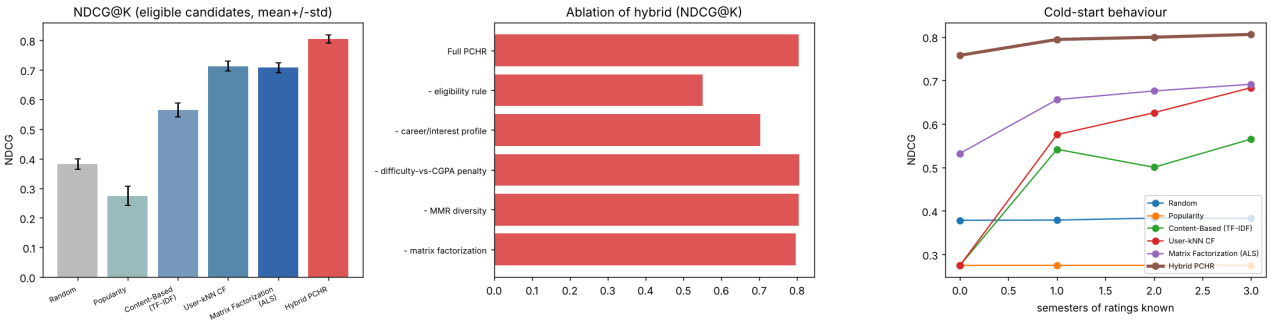


Figure 1. Left: NDCG@5 in the fair setting with standard deviation. Centre: ablation of the hybrid. Right: cold-start behaviour as fewer ratings are known.

8.3 Rating-prediction error

Model	RMSE	MAE
Popularity (damped item mean)	1.112	0.931
User-kNN CF	1.024	0.844
Matrix Factorization (ALS)	1.002	0.805
Random, Content-Based, Hybrid	n/a	n/a

Table 5. Error on held-out ratings (lower is better). Models whose score is a similarity or blended ranking score do not predict a 1 to 5 rating, so RMSE does not apply.

8.4 Ablation of the hybrid

Variant	Precision	Recall	NDCG	Topic spread	Eligible
Full PCHR	0.366	0.924	0.805	0.641	1.00
- eligibility rule	0.258	0.632	0.550	0.405	0.45
- career/interest profile	0.328	0.842	0.704	0.682	1.00
- difficulty-vs-CGPA penalty	0.366	0.923	0.806	0.639	1.00
- MMR diversity	0.367	0.925	0.805	0.635	1.00
- matrix factorization	0.363	0.915	0.797	0.637	1.00

Table 6. Ablation: one component removed at a time.

8.5 Diversity versus accuracy (MMR weight gamma)

gamma	0	0.1	0.2	0.3	0.5
NDCG	0.805	0.805	0.804	0.804	0.796
Precision	0.367	0.367	0.366	0.366	0.362
Topic spread	0.635	0.638	0.642	0.648	0.672

Table 7. Raising gamma increases topic spread at almost no cost in accuracy. The default is gamma = 0.15.

8.6 Cold start

Semesters of ratings known	Random	Popularity	Content	kNN	MF	Hybrid
0	0.378	0.275	0.275	0.275	0.532	0.758
1	0.379	0.275	0.542	0.575	0.656	0.794
2	0.384	0.275	0.500	0.626	0.676	0.799
3	0.383	0.275	0.565	0.683	0.691	0.806

Table 8. NDCG@5 when only the first n semesters of ratings are known (the transcript, and so eligibility, is always known).

8.7 Interpretation

- **The hybrid is best.** In the fair setting it reaches NDCG 0.805 against 0.713 for kNN and 0.708 for matrix factorization, and it won on every dataset.
- **What drives the gain.** Removing the eligibility rule drops NDCG to 0.55 and removing the career/interest profile drops it to 0.704, about the level of kNN. The difficulty penalty, MMR and matrix factorization change NDCG by less than 0.01. The advantage over kNN therefore comes mainly from the profile information and the rules, not from blending alone.
- **kNN and matrix factorization are tied** for ranking (0.713 against 0.708, within one standard deviation), while matrix factorization has the lowest rating error.
- **Random is a strong floor.** With only about 9.5 eligible courses and 5 slots, guessing scores NDCG 0.38. Popularity falls below Random because it repeats the same widely taken courses for everyone.

- **Recall is high** (0.92) for the same reason: the pool is small. Results should be read relative to Random.
- **Diversity.** The hybrid's topic spread (0.641) is below Random's (0.75) because personalised lists concentrate on a student's interests. MMR with TF-IDF similarity raises spread only slightly (0.635 to 0.672 at gamma 0.5).
- **Cold start.** With no ratings the hybrid keeps NDCG 0.758 using profile and rules, while rating-based models fall to 0.275 to 0.532. Content and kNN equal Popularity at zero ratings because all their scores tie and popularity breaks the tie.

9. Demonstration

The command-line program prints every model's top five for a chosen student, the hybrid's explanations and the courses the student actually took next. A second mode asks for a new student's profile. Excerpt for student S0007 (IT, CGPA 7.84, goal Research / Higher Studies, interests Data and Theory, entering semester 4):

```
HYBRID PCHR - with explanations
  1. CS206  Python for Data Science               [Data, year 2, difficulty 2]
       - fits your goal/interests (Research / Higher Studies; Data)
       - similar to courses you rated highly: Introduction to Data Analysis, ...
       - prerequisites done: CS101, CS107
       - difficulty 2/5 suits your CGPA
  2. CS211  Statistical Methods for Data Analysis [Data, year 2, difficulty 3]
       - fits your goal/interests (Research / Higher Studies; Data)
       - prerequisites done: CS104
  3. CS207  Introduction to Artificial Intelligence
  4. CS108  Cyber Hygiene and Ethics
  5. CS210  Introduction to Cryptography

Actually took next semester: CS211(rated 4), CS106(1), CS206(rated 5), CS210(3), CS108(2)
```

The hybrid placed the two relevant courses (CS206 and CS211, rated 5 and 4) at ranks 1 and 2, giving precision 0.4, recall 1.0 and NDCG 1.0 for this student.

10. Discussion, limitations and ethics

10.1 Limitations

- **Synthetic data.** Results hold under our stated rules and do not prove accuracy on real students. Because the generator uses interests and career goals, the profile terms are informative by construction. We reduced this by adding noise (declared interests are wrong for one of two topics in 25 percent of students) but the effect on real data is unknown.
- **Unequal information.** The hybrid uses career goal and declared interests, which the baselines are not given. The ablation shows that without them it performs about like kNN.
- **Small eligible pool.** About 9.5 eligible courses per student make Random a strong baseline and recall values high.
- **Little effect from some parts.** The difficulty penalty and MMR add under 0.01 NDCG here.
- **Statistics.** Five datasets, no formal significance test, and kNN and matrix factorization hyper-parameters were set in advance rather than tuned.
- **Evaluation design.** Neighbours with more semesters may carry information from after the target student's test semester; we treat students as different cohorts, but a strictly time-stamped evaluation would remove this. Explicit matrix factorization ignores that ratings exist only for courses students chose.

10.2 Ethics and fairness

Grades and ratings are sensitive and must be protected. Recommenders can narrow students into one topic and can create feedback loops that change what students choose. The difficulty penalty could steer weaker students away from challenging courses, so difficulty is shown in each explanation, the penalty is mild, and the output is advisory. Before real deployment, quality and coverage should be checked across branches and CGPA bands.

11. Conclusion and future work

We built and compared five recommendation approaches for course selection under a common, leakage-aware protocol, and designed a hybrid that respects prerequisites, uses career and interest information, and explains its advice. On synthetic data it improved NDCG@5 from 0.713 to 0.805 over the best single model on every dataset, and remained useful with no ratings. The ablation shows honestly that the rules and profile drive the gain, while blending, the difficulty penalty and diversity re-ranking contribute little on this data.

Future work: validate on real enrolment data and with advisers; use implicit-feedback factorization [7] and learn-to-rank over the component scores; use a topic-aware similarity in MMR; tune kNN and matrix factorization; add credit limits and timetable clashes; run formal significance tests with more seeds; and add automated tests.

12. Team contributions

Member	Role	Parts of the code and report
1	Problem formulation and data	course catalogue, data generator, dataset class, eligibility rule and temporal split; Sections 1 and 3
2	Baselines, content-based filtering and demo	Random, Popularity, TF-IDF model, command-line demo; Sections 5.1, 5.2 and 9
3	Collaborative filtering and matrix factorization	user-kNN, ALS matrix factorization; Sections 5.3 and 5.4
4	Hybrid improvement	PCHR scoring, tuning, MMR, explanations, ablation and gamma sweep; Section 6
5	Evaluation and integration	metrics, evaluation loop, comparison tables and charts, report; Sections 4, 7, 8 and 10

Table 9. Division of work.

References

[1] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom and J. Riedl, "GroupLens: An open architecture for collaborative filtering of netnews," in Proc. ACM CSCW, 1994.

[2] Y. Koren, R. Bell and C. Volinsky, "Matrix factorization techniques for recommender systems," IEEE Computer, vol. 42, no. 8, pp. 30-37, 2009.

[3] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Information Processing and Management, vol. 24, no. 5, pp. 513-523, 1988.

[4] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in Proc. ACM SIGIR, 1998.

[5] K. Jarvelin and J. Kekalainen, "Cumulated gain-based evaluation of IR techniques," ACM Transactions on Information Systems, vol. 20, no. 4, pp. 422-446, 2002.

[6] F. Ricci, L. Rokach and B. Shapira (eds.), Recommender Systems Handbook, 2nd ed., Springer, 2015.

[7] Y. Hu, Y. Koren and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in Proc. IEEE ICDM, 2008.

[8] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2020.

[9] R. Burke, "Hybrid recommender systems: Survey and experiments," User Modeling and User-Adapted Interaction, vol. 12, no. 4, pp. 331-370, 2002.

[10] W. Kool, H. van Hoof and M. Welling, "Stochastic beams and where to find them: The Gumbel-top-k trick for sampling sequences without replacement," in Proc. ICML, 2019.

Appendix: running the code

The complete program is the single file `course_recommender.py`. Install the libraries once with `pip install numpy pandas scikit-learn matplotlib`, then:

```
python course_recommender.py compare                # all tables and results/comparison.png (about 1 minute)
python course_recommender.py compare --seeds 2 --k 3   # quicker run, different list length
python course_recommender.py demo --student S0007      # every model's top 5 with explanations
python course_recommender.py demo --new                # cold-start demo with your own profile
python course_recommender.py generate                  # write data/courses.csv, students.csv, ratings.csv
```

Parameter	Value	Location in the code
List length K	5	command line, --k
kNN neighbours / shrinkage	30 / 2.0	UserKNNRecommender.__init__
MF factors / regularization / iterations	8 / 4.0 / 15	MatrixFactorization.__init__
MMR weight gamma	0.15	DEFAULT_WEIGHTS
Relevance threshold	4	REL_THRESHOLD
Tuning grid	96 combinations	tune_hybrid

Table 10. Main parameters.